Mnemox

Give IBM Bob a memory. Make your knowledge compound.

TEAM BOBJECTIFLUNE · SOFTWARE · 2026 IBMER WATSONX CHALLENGE

The problem. Every IBM Bob session starts **amnesiac.** It re-reads your repo and re-answers last week's question — you pay Bobcoins *and* your own time to relearn what Bob already knew.

Knowledge should compound — not restart.

The fix isn't a shorter prompt. It's a **memory**. Mnemox implements Andrej Karpathy's **LLM-Wiki** pattern — but **native to IBM Bob**, as two modes, no plugin, no MCP server, no external service. Knowledge is written once and *retrieved* thereafter, so every session launches from accumulated altitude instead of the ground.

WITHOUT A MEMORY

2,000 lines

Session 1 reads the repo. Session 10 reads the same 2,000 lines *again*. Bob re-derives the same decisions every time.

WITH MNEMOX

~200 lines

Session 10 retrieves a curated digest from the knowledge base. Grounded answer, a fraction of the context, budget freed for real work.

BobjectifLune — aim at the impossible.

BobjectifLune = **Bob** + *Objectif Lune* (Destination Moon): reach what looks impossible through curiosity, involvement, a fail-fast mindset, and resiliency — with Bob as the craft and the knowledge base as the **flight log**. Each value maps to something real in this project:

Curiosity
Reframed cost as a *memory* problem: "why re-pay for what Bob already knew?"

Fail-fast
Shipped a wrong 68.96% figure — caught it fast, retracted it publicly.

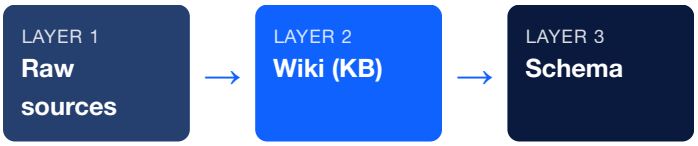
Resiliency
Rebuilt a manifest-backed harness + CI that blocks any unprovenanced number.

Involvement
Team-shared KB in Git — one member's knowledge becomes everyone's.

The thesis in one line: On the way to the impossible, you cannot afford to forget what you already learned. Mnemox is the memory IBM Bob was missing.

Two native Bob modes. Zero plugins.

- 🧠 **Knowledge Builder** — a living KB (concepts, guides, references, research) with templates, cross-references, and a self-updating index.
- 🔍 **Repo Analyzer** — 7 Bash scripts produce dated ~200-line digests (scan, deps, metrics, security, coverage, git, docs) that Bob cites instead of raw source.
- Works in **Bob IDE** (mode picker — zero install) and **Bob Shell CLI**. One command scaffolds a new workspace in <30s.



Karpathy's three layers, native to Bob: raw sources → curated wiki → a stable schema (mode + `AGENTS.md`) that makes Bob disciplined and cacheable.

Six token-economy taxes — each has a Bash-KB fix (no Python required).

WASTE	HOW THE KB MANAGER REMOVES IT
Catalog tax — MCP re-sends its tool list every turn	Native Bob mode — no plugin, no MCP
Payload tax — a 2,000-line file for 20 relevant lines	Scripts summarise; the KB stores digests you *cite*
Compression trap — stripping meaning raises effective cost	Templates preserve rationale, real names, cross-refs
Short-thread tax — turn 15 re-pays 14 turns of history	Knowledge persists in KB + `save_memory`; a fresh thread retrieves
Cache misses — reworded prefixes	Fixed mode + `AGENTS.md` + stable layout = cacheable prefix
Verbose output	Bounded artifacts — templates, `INDEX.md`, reports — not essays

The numbers — separated, labelled, provenanced.

20.0%
mean token compression
N=183 real docs · 95% CI [18.9–21.2%] · null test 0.73% · manifest-backed

20→92%
effective saving w/ cache
Scales with your repetition rate (0%→90% repeat). We publish the model, not one flattering number.

≈60%
fewer Bobcoins / iteration
hcd-at-its-core: 94 demos, 15 masterclasses. Session-readout, single-project observation.

The reinvestment story is the real result. The ≈60% isn't the impact — it's the mechanism. Every Bobcoin recovered on **hcd-at-its-core** was reinvested into deeper failure modes, richer topologies, and more entropic scenarios across 94 demos that would otherwise have been unaffordable to build.

Not RAG. Not a plugin. A compounding memory that won't overclaim.

	RAG	MCP SERVER	MNEMOX (BOB-NATIVE KB)
Rediscovers from scratch every query	Yes	Yes	No — retrieves
Catalog tax every turn	No	Yes	No
Knowledge accumulates & compounds over time	No	No	Yes
Works natively in Bob IDE (zero install)	Depends	Depends	Yes
Infrastructure required	Vector DB + embeddings + pipeline	Server + catalog	Git only

Wins on trust.

This project caught its *own* fabricated **68.96% "VALIDATED"** figure, formally retracted it, and built CI that blocks any unprovenanced number. In an era of confidently-wrong AI, **a productivity tool that refuses to overclaim its own productivity** is the differentiator.

STRIDE threat model · bandit SAST · passing null test · manifest-backed validation · formal retraction on file · every claim labelled at its true confidence level.

Adopt in one click.

- **Bob IDE:** mode picker → 🧠 Mnemox Knowledge Builder. Zero install, zero CLI.
- **Bob Shell CLI:** one alias, one command.
- Plain Markdown in Git — no proprietary format, no lock-in. **MIT-licensed**; any IBM team can clone, init, and start compounding.
- Deploy to a new workspace in a single command; the whole team activates from the picker.

The frontier. When re-derivation cost approaches zero, the budget freed from repetition funds depth, breadth, and deeper agent teams — it moves the frontier of what's affordable to attempt with Bob. Bob got us off the ground; the knowledge base is the flight log that keeps us climbing.

The impossible, made repeatable.

What's next. A domain-specific starter KB for watsonx.data Tiger Teams, so every South EMEA engagement launches from accumulated expertise instead of zero — turning recurring re-derivation cost into a one-time, team-shared institutional asset.